MuleSoft to Spring Boot Migrator

Complete Technical Documentation

A Beginner-Friendly Guide

Version: 1.0

Date: March 2026

Author: Harinadh

Technology Stack: Python Flask + Spring Boot 3.2

Table of Contents

1. Introduction

1.1 What is This Tool?

The MuleSoft to Spring Boot Migrator is a web application that automatically converts MuleSoft 4 integration projects into Spring Boot 3.2 Java applications. Think of it as a "translator" that reads your MuleSoft XML configuration files and produces a complete, ready-to-run Java project.

1.2 Why Would You Need This?

Many organizations use MuleSoft (owned by Salesforce) for building APIs and integrations. However, there are several reasons why a company might want to move to Spring Boot:

- **Cost Reduction:** MuleSoft licenses can be expensive. Spring Boot is free and open-source.
- **Control:** With Spring Boot, you own and control your entire codebase.
- **Flexibility:** Spring Boot has a massive ecosystem of libraries and tools.
- **Talent Pool:** More developers know Java/Spring Boot than MuleSoft.
- **Performance:** Spring Boot applications can be highly optimized for your specific use case.

1.3 What Does It Actually Do?

Here is exactly what happens when you use this tool:

1. You upload your MuleSoft XML files (the files that define your APIs and integrations)
2. The tool reads and understands every element in those XML files
3. It identifies all connectors (HTTP, Database, JMS, Kafka, etc.)
4. It converts MuleSoft flows into Java classes with Spring Boot annotations
5. It translates DataWeave expressions into equivalent Java code
6. It generates a complete Maven project with all dependencies
7. It creates configuration files (application.properties) with correct settings
8. Optionally, an AI model reviews the generated code for quality and suggestions
9. You download a ZIP file containing a complete, runnable Spring Boot project

1.4 Who Is This Documentation For?

This documentation is written for everyone, from complete beginners to experienced developers. Every technical term is explained. Every step includes detailed instructions. You do NOT need prior experience with MuleSoft, Spring Boot, or programming to understand this guide.

2. Key Concepts for Beginners

2.1 What is MuleSoft?

MuleSoft is a platform for building application programming interfaces (APIs) and integrations. It allows businesses to connect different software systems together. For example, connecting your website to a database, or connecting your payment system to your inventory system. MuleSoft uses XML configuration files to define how data flows between systems. These XML files contain "flows" which are like step-by-step instructions.

2.2 What is Spring Boot?

Spring Boot is a popular Java framework for building web applications and APIs. It is open-source (free to use) and is the most widely used Java framework in the world. Spring Boot makes it easy to create production-ready applications with minimal configuration.

2.3 What is an API?

An API (Application Programming Interface) is a way for two software programs to talk to each other. Think of it like a waiter in a restaurant: you (the client) tell the waiter (the API) what you want, the waiter goes to the kitchen (the server), and brings back your food (the response).

2.4 What is Docker?

Docker is a tool that packages your application and everything it needs into a "container." Think of a container like a shipping container: no matter what ship (computer) carries it, the contents are always the same and always work.

2.5 What is DataWeave?

DataWeave is MuleSoft's own programming language for transforming data. For example, if you receive customer data in one format but need to send it in a different format, DataWeave handles that transformation. Our tool converts DataWeave code into equivalent Java code.

2.6 What is an LLM (AI Model)?

LLM stands for Large Language Model. These are AI systems (like ChatGPT, Claude, Gemini) that can understand and generate text, including code. Our tool can optionally send the generated Spring Boot code to an LLM for review, getting suggestions for improvements.

Common API Terms:

Term	What It Means	Example
Endpoint	A specific URL that accepts requests	/api/customers

Term	What It Means	Example
GET	A request to retrieve data	Get list of all customers
POST	A request to create new data	Create a new customer
PUT	A request to update existing data	Update customer address
DELETE	A request to remove data	Delete a customer record
JSON	A text format for sending/receiving data	{"name": "John", "age": 30}
XML	Another text format (used by MuleSoft)	<name>John</name>

3. System Requirements

3.1 Minimum Requirements

Requirement	Minimum	Recommended
Operating System	macOS 10.15+, Windows 10+, Linux	macOS 12+ or Ubuntu 22.04+
Python	3.9 or higher	3.12
RAM	4 GB	8 GB or more
Disk Space	500 MB	2 GB (for Docker)
Browser	Any modern browser	Chrome or Firefox (latest)
Internet	Required for LLM validation	Broadband connection

3.2 Optional Requirements

Component	When Needed	Purpose
Docker	Production deployment	Run the app in a container
LLM API Key	AI code validation	Review generated code quality
Java 17+	To run generated project	Build the Spring Boot output
Maven 3.9+	To build generated project	Compile and package the Java project

4. Installation Guide

4.1 Method 1: Local Installation (Recommended for Development)

Step 1: Verify Python is Installed

Open your terminal and type:

```
python3 --version
```

You should see something like "Python 3.9.6" or higher. If you get an error, install Python from python.org.

Step 2: Navigate to the Project Folder

```
cd "/path/to/mulesoft-to-springboot-migrator/backend"
```

Step 3: Install Dependencies

This installs all the Python packages the tool needs:

```
pip3 install -r requirements.txt
```

Step 4: Start the Server

For development:

```
python3 app.py
```

For production:

```
python3 -m gunicorn -c gunicorn.conf.py app:app
```

Step 5: Open in Browser

```
http://localhost:5000
```

> **TIP:** You should see the MuleSoft to Spring Boot Migrator interface with a dark-themed UI.

4.2 Method 2: Docker Installation (Recommended for Production)

Step 1: Install Docker Desktop from docker.com

Step 2: Create Environment File

```
cp .env.example .env
# Edit .env with your favorite text editor
```

Step 3: Start with Docker Compose

```
# Basic start
docker compose up -d
```

```
# With Nginx reverse proxy
docker compose --profile with-nginx up -d
```

Step 4: Verify

```
curl http://localhost:5000/api/health
# {"status": "ok", "env": "production"}
```

4.3 Stopping the Server

Method	Command
Local (Gunicorn)	kill $(pgrep -f 'gunicorn.*app:app')
Local (Flask dev)	Press Ctrl+C in the terminal
Docker	docker compose down

5. Quick Start Tutorial

Let's walk through your very first migration. This tutorial takes about 5 minutes.

Step 1: Open the Application

Start the server (see Chapter 4) and open http://localhost:5000 in your browser. You will see a dark-themed interface split into two panels: Input (left) and Output (right).

Step 2: Load Sample Data

Click the "Load Sample" button in the top-right corner. This fills in a sample MuleSoft XML configuration that demonstrates HTTP endpoints, database queries, and DataWeave transformations.

Step 3: Click Migrate to Spring Boot

Click the large blue "Migrate to Spring Boot" button at the bottom of the left panel. The tool will process the XML and generate a complete Spring Boot project in 1-2 seconds.

Step 4: Explore the Output

The right panel shows three tabs: Generated Files (file tree with all Java files), Summary (migration statistics), and AI Review (if LLM validation was enabled).

Step 5: Download the Project

Click "Download ZIP" to download the complete Spring Boot project. Unzip it, open in your Java IDE (IntelliJ, Eclipse, VS Code), and run it!

> **TIP:** The generated project includes a Dockerfile and docker-compose.yml, so you can also run it with "docker compose up -d" without installing Java locally!

6. Understanding the User Interface

6.1 Overall Layout

Section	Location	Purpose
Header Bar	Top of page	Logo, title, Load Sample and Clear buttons
Input Panel	Left side	MuleSoft XML, DataWeave, Settings, AI Validation
Output Panel	Right side	Generated code, Summary, AI Review
Status Bar	Bottom	Status messages and progress
Loading Overlay	Center	Animated spinner during processing

6.2 Input Panel - MuleSoft XML Tab

- **Upload Zone:** Drag and drop XML files or click to browse. Accepts .xml, .raml, .yaml, .yml
- **Multiple Files:** Upload multiple files. Each appears as a chip with filename and size
- **Code Editor:** Syntax-highlighted editor (CodeMirror) for pasting XML directly
- **Preview:** Click any uploaded file chip to preview its contents

6.3 Input Panel - DataWeave Tab

- **Script List:** Shows all DataWeave scripts as tabs
- **Add Script:** Click "+" to create a new named DataWeave script
- **Convert:** Click to see the Java equivalent of just this script

6.4 Input Panel - Settings Tab

Setting	Default	Description
Project Name	my-spring-app	Name for your Spring Boot project
Group ID	com.example	Java package namespace
Java Version	17	Target Java version (17 or 21 recommended)

6.5 Input Panel - AI Validation Tab

- **Enable/Disable Toggle:** Turn AI validation on or off
- **Provider Dropdown:** Select AI provider (Anthropic, OpenAI, Google, etc.)
- **Model Dropdown:** Choose a specific model
- **API Key Field:** Enter your API key (never stored on disk)
- **Base URL:** Only needed for Ollama (local AI)
- **Test Connection:** Verify your API key works before migrating

6.6 Output Panel Tabs

- **Generated Files:** Hierarchical file tree of all generated files. Click to view with syntax highlighting.
- **Summary:** Statistics - flows converted, connectors found, dependencies, warnings
- **AI Review:** Score (1-10), issues by severity, improvements, security issues, best practices

7. How the Migration Works

The migration follows a pipeline of 6 steps. Understanding this pipeline helps you understand the output and troubleshoot any issues.

Step 1: Parse (parser.py)

The XML parser reads each MuleSoft XML file and extracts all meaningful elements: flows, sub-flows, connectors, configurations, error handlers, DataWeave transformations, batch jobs, and more. It understands 30+ MuleSoft namespaces.

Step 2: Merge (app.py)

If multiple XML files were provided, the parsed results are merged into a single unified structure. Duplicate flows are detected and the first occurrence is kept with a warning.

Step 3: Map Connectors (connector_mapper.py)

Each detected connector is mapped to its Spring Boot equivalent. This determines Maven dependencies, Spring annotations, and configuration properties.

Step 4: Convert Flows (flow_converter.py)

Each MuleSoft flow is converted to a Java class. HTTP listeners become REST controllers, schedulers become @Scheduled methods, message listeners become JMS/Kafka/AMQP listeners.

Step 5: Generate Project (spring_generator.py)

The complete Spring Boot project is assembled: pom.xml, Application class, configuration classes, application.properties, exception classes, Dockerfile, and docker-compose.yml.

Step 6: Validate (llm_validator.py, Optional)

If AI validation is enabled, the generated code is sent to an LLM for review. The AI scores the code from 1-10 and provides actionable feedback.

8. Supported MuleSoft Components

8.1 Supported Connectors (30+)

A "connector" in MuleSoft is a pre-built module for connecting to external systems:

MuleSoft Connector	Purpose	Spring Boot Equivalent
HTTP Listener	Receives API requests	Spring Web (@RestController)
HTTP Request	Sends API requests	RestTemplate / WebClient
Database	MySQL, Oracle, PostgreSQL, MSSQL	Spring Data JPA / JdbcTemplate
JMS	Java message queues	JmsTemplate / @JmsListener
AMQP	RabbitMQ messaging	RabbitTemplate / @RabbitListener
Kafka	Event streaming	KafkaTemplate / @KafkaListener
VM	In-memory messaging	Spring Events (built-in)
File	Local file operations	java.nio.file.Files
SFTP	Secure file transfer	Spring Integration SFTP
FTP	File transfer protocol	Spring Integration FTP
Email	IMAP, POP3, SMTP	Spring Mail
SOAP/WS	SOAP web services	Spring Web Services
Salesforce	CRM integration	RestTemplate + OAuth
Amazon S3	Cloud file storage	AWS SDK v2 S3Client
Amazon SQS	Cloud message queue	Spring Cloud AWS
MongoDB	NoSQL document DB	Spring Data MongoDB
Redis	In-memory cache/store	Spring Data Redis
Elasticsearch	Search engine	Spring Data Elasticsearch
Batch	Batch job processing	Spring Batch
OAuth/Security	Authentication	Spring Security
Validation	Input validation	Jakarta Bean Validation

8.2 Supported Processors (50+)

Core Logic Processors:

MuleSoft Processor	What It Does	Java Equivalent
choice	If/else branching	if-else statements
for-each	Loop through items	Java Stream .forEach()
parallel-for-each	Process in parallel	parallelStream()
scatter-gather	Run tasks simultaneously	CompletableFuture.allOf()
try	Handle errors	try-catch block
until-successful	Retry operations	@Retryable annotation
async	Background processing	@Async annotation
flow-ref	Call another flow	Service method call

Connector-Specific Processors:

Processor	What It Does	Java Equivalent
http:request	Send HTTP request	restTemplate.exchange()
db:select	Query database	jdbcTemplate.query()
db:insert	Add database record	jdbcTemplate.update()
db:update	Modify records	jdbcTemplate.update()
db:delete	Remove records	jdbcTemplate.update(DELETE)
jms:publish	Send message to queue	jmsTemplate.convertAndSend()
file:read	Read a file	Files.readString(Path)
file:write	Write a file	Files.writeString(Path, content)

8.3 Error Type Mapping (100+ types)

MuleSoft Error	Java Exception	HTTP Status
HTTP:NOT_FOUND	ResourceNotFoundException	404
HTTP:BAD_REQUEST	BadRequestException	400
HTTP:UNAUTHORIZED	UnauthorizedException	401
HTTP:TIMEOUT	SocketTimeoutException	408
DB:CONNECTIVITY	DataAccessResourceFailureException	503
VALIDATION:INVALID_*	ConstraintViolationException	400

9. DataWeave to Java Conversion

DataWeave is MuleSoft's proprietary data transformation language. This tool converts DataWeave 2.0 expressions into equivalent Java code using Java Streams.

9.1 Collection Operations

DataWeave	What It Does	Java Equivalent
map	Transform each item	.stream().map(...).collect(toList())
filter	Keep matching items	.stream().filter(...).collect(toList())
reduce	Combine into one value	.stream().reduce(...)
flatMap	Map and flatten	.stream().flatMap(...)
groupBy	Group by key	.stream().collect(groupingBy(...))
orderBy	Sort items	.stream().sorted(...)
distinctBy	Remove duplicates	.stream().distinct()
sizeOf	Count items	.size()
isEmpty	Check if empty	.isEmpty()

9.2 String Operations

DataWeave	Java Equivalent
upper()	.toUpperCase()
lower()	.toLowerCase()
trim()	.trim()
contains()	.contains()
replace ... with ...	.replace(old, new)
splitBy()	.split()
++ (concatenation)	+ (string concat)

9.3 Type Coercion

DataWeave	Java Equivalent
as String	String.valueOf(x)
as Number	Double.parseDouble(x)
as Boolean	Boolean.parseBoolean(x)
as Date	LocalDate.parse(x)

DataWeave	Java Equivalent
as DateTime	LocalDateTime.parse(x)
value default "fallback"	value != null ? value : "fallback"

9.4 Example Conversion

DataWeave:

```
%dw 2.0
output application/json
---
payload map (item) -&gt; {
fullName: item.firstName ++ " " ++ item.lastName,
age: item.age as Number
}
```

Generated Java:

```
List&lt;Map&lt;String,Object&gt;&gt; result =
((List&lt;Map&lt;String,Object&gt;&gt;) payload).stream()
.map(item -&gt; {
Map&lt;String,Object&gt; obj = new LinkedHashMap&lt;&gt;();
obj.put("fullName", item.get("firstName") + " " +
item.get("lastName"));
obj.put("age", Double.parseDouble(
String.valueOf(item.get("age"))));
return obj;
}).collect(Collectors.toList());
```

10. AI-Powered Code Validation

10.1 Supported AI Providers

Provider	Models	Pricing	Best For
Anthropic Claude	Sonnet 4, 3.5 Sonnet, Opus, Haiku	Paid	Best code review quality
OpenAI GPT	GPT-4o, GPT-4 Turbo, 4o-mini, o3-mini	Paid	Wide availability
Google Gemini	2.5 Pro, 2.0 Flash, 1.5 Pro	Free tier available	Cost-effective
DeepSeek	Chat, Coder, Reasoner	Very affordable	Budget-friendly
Groq	Llama 3.3 70B, Mixtral, Llama 3.1 8B	Free!	Fast and free
Ollama (Local)	CodeLlama, Llama 3, Mistral, Qwen	Free (local)	Complete privacy

10.2 Setup Steps

1. Go to the "AI Validation" tab in the input panel
2. Toggle the switch to enable AI validation
3. Select your preferred provider from the dropdown
4. Choose a model (speed vs quality tradeoff)
5. Enter your API key (get from provider's website)
6. Click "Test Connection" to verify
7. Click "Migrate" - AI review is included automatically

10.3 Understanding the Score

Score	Color	Meaning
8-10	Green	Excellent - Production-ready with minor suggestions
6-7	Yellow	Good - Works but has notable improvements
4-5	Orange	Fair - Significant issues to address
1-3	Red	Poor - Major problems need fixing

10.4 Getting Free API Keys

Provider	How to Get	Free Tier
Groq	Sign up at console.groq.com	Completely free with generous limits
Google Gemini	Sign up at aistudio.google.com	Free tier with rate limits

Provider	How to Get	Free Tier
Ollama	Install from ollama.com (no key needed)	Completely free, runs locally

11. Multi-File Migration

Real MuleSoft projects typically have multiple XML files. This tool handles that seamlessly.

11.1 How It Works

1. Upload multiple XML files using drag-drop or file browser
2. Each file appears as a chip with name and size
3. Click any chip to preview the file contents
4. When you click "Migrate", ALL files are processed together
5. Each file is parsed independently, then results are merged into one project

11.2 Merge Rules

Component	Merge Rule	On Conflict
Flows	Combined from all files	Duplicate names: first kept + warning
Sub-Flows	Combined from all files	Duplicate names: first kept + warning
Configurations	Combined by name	Duplicate names: first kept
Properties	Merged into single dict	Same key: last value wins
Connectors	Union of all detected	No conflicts (sets)
Error Handlers	All collected	No conflicts (all kept)

12. Generated Spring Boot Project

12.1 Project Structure

```
my-spring-app/
+-- pom.xml # Maven build file
+-- Dockerfile # Container image
+-- docker-compose.yml # Container orchestration
+-- src/main/java/.../
| +-- Application.java # Main entry point
| +-- controller/ # REST controllers
| +-- service/ # Business logic
| +-- config/ # Configuration classes
| +-- exception/ # Custom exceptions
+-- src/main/resources/
| +-- application.properties # App configuration
+-- src/test/java/.../
+-- ApplicationTests.java # Unit tests
```

12.2 Key Generated Files

File	Purpose	When Generated
pom.xml	Maven dependencies	Always
Application.java	Main class with annotations	Always
*Controller.java	REST endpoints	When HTTP flows exist
*Service.java	Business logic	When flows have logic
*Listener.java	Message listeners	When messaging used
SchedulingConfig.java	Enable scheduling	When schedulers found
JmsConfig.java	JMS configuration	When JMS used
KafkaConfig.java	Kafka configuration	When Kafka used
SecurityConfig.java	OAuth2/JWT security	When OAuth used
application.properties	All settings	Always
Dockerfile	Docker build definition	Always
docker-compose.yml	Services (DB, MQ, etc.)	Always

12.3 Running the Generated Project

Option A: With Maven (requires Java 17+ and Maven):

```
cd my-spring-app
mvn spring-boot:run
```

Option B: With Docker:

```
cd my-spring-app
docker compose up -d
```

TIP: The docker-compose.yml includes all required services (MySQL, RabbitMQ, Kafka, etc.) that your migrated app needs. Just run "docker compose up" and everything starts!

13. API Reference

The migrator exposes HTTP API endpoints you can use programmatically (curl, Postman, etc.).

GET /api/health

Check server health

Request:

```
No body needed
```

Response:

```
{"status": "ok", "env": "production"}
```

GET /api/llm/providers

Get all available AI providers and models

Request:

```
No body needed
```

Response:

```
{"anthropic": {"name": "...", "models": [...]}, ...}
```

POST /api/migrate

Main migration endpoint - accepts XML, returns Spring Boot project

Request:

```
{"muleXmlFiles": [...], "projectName": "...",
"llmConfig": {"enabled": true, ...}}
```

Response:

```
{"success": true, "files": {...},
"summary": {...}, "llmValidation": {...}}
```

POST /api/validate

Standalone AI code review

Request:

```
{"files": {...}, "summary": {...},
"llmConfig": {"provider": "...", ...}}
```

Response:

```
{"success": true, "validation": {"overallScore": 7, ...}}
```

POST /api/migrate/download

Download generated project as ZIP

Request:

```
{"files": {...}, "projectName": "my-app"}
```

Response:

```
Binary ZIP file
```

POST /api/convert/dataweave

Convert DataWeave to Java

Request:

```
{"script": "%dw 2.0\n..."}
```

Response:

```
{"success": true, "result": {"java_code": "...", ...}}
```

14. Production Deployment

14.1 Architecture

Layer	Technology	Purpose
Reverse Proxy	Nginx	SSL, rate limiting, caching, security headers
App Server	Gunicorn	Multi-worker Python server for concurrent requests
Application	Flask	The actual migrator web application and API

14.2 Gunicorn Settings

Setting	Default	Description
workers	CPU cores x 2 + 1	Worker processes for requests
worker_class	gthread	Thread-based (good for LLM I/O)
threads	4	Threads per worker
timeout	120 seconds	Max request time (LLM calls)
bind	0.0.0.0:5000	Listen address and port

14.3 Nginx Features

- **Rate Limiting:** 2 req/s for migrations, 10 req/s for other API endpoints
- **Gzip Compression:** Level 6 for text, JSON, JavaScript, SVG
- **Security Headers:** X-Frame-Options, CSP, X-XSS-Protection, Referrer-Policy
- **Static Caching:** 7-day browser cache for CSS/JS files
- **SSL/TLS:** HTTPS template included (requires certificates)
- **Request Limit:** 50 MB maximum upload size

14.4 Docker Commands

Command	What It Does
docker compose up -d	Start the application
docker compose --profile with-nginx up -d	Start with Nginx proxy
docker compose logs -f app	View real-time logs
docker compose down	Stop all containers
docker compose ps	Show container status

14.5 Environment Variables

Variable	Default	Description
FLASK_ENV	production	Flask environment mode
PORT	5000	Application port
SECRET_KEY	(must set)	Session security key
CORS_ORIGINS	*	Allowed API origins
GUNICORN_WORKERS	4	Worker processes
ANTHROPIC_API_KEY	(empty)	Anthropic Claude API key
OPENAI_API_KEY	(empty)	OpenAI GPT API key
GOOGLE_API_KEY	(empty)	Google Gemini API key
GROQ_API_KEY	(empty)	Groq API key (free)

IMPORTANT: API keys can be set in the .env file (for Docker) or entered directly in the UI. The UI never stores API keys on disk for security.

15. Configuration Reference

15.1 Project Files

File	Location	Purpose
app.py	backend/	Main Flask application with routes
gunicorn.conf.py	backend/	Production server configuration
requirements.txt	backend/	Python package dependencies
parser.py	backend/migrator/	XML parser (30+ namespaces)
connector_mapper.py	backend/migrator/	Connector to Spring mapper
flow_converter.py	backend/migrator/	Flow to Java converter
spring_generator.py	backend/migrator/	Project generator
dataweave_converter.py	backend/migrator/	DataWeave to Java converter
llm_validator.py	backend/migrator/	LLM integration (6 providers)
index.html	backend/templates/	Web application HTML
app.js	backend/static/	Frontend JavaScript
style.css	backend/static/	UI stylesheet
Dockerfile	project root	Docker image definition
docker-compose.yml	project root	Container orchestration
nginx.conf	nginx/	Nginx configuration

IMPORTANT: API keys are NEVER stored in localStorage, cookies, or anywhere on disk. You must re-enter your API key each session. This is a deliberate security measure.

16. Troubleshooting Guide

16.1 Installation Issues

pip command not found

On macOS, use pip3 instead of pip:

```
pip3 install -r requirements.txt
```

gunicorn command not found

Use python3 -m gunicorn instead:

```
python3 -m gunicorn -c gunicorn.conf.py app:app
```

Address already in use (port 5000)

Kill the existing process:

```
kill $(lsof -t -i :5000)
```

lxml installation fails

Install system libraries first:

```
brew install libxml2 libxslt # macOS
```

16.2 Migration Issues

Issue	Cause	Solution
Invalid XML error	XML syntax errors	Validate XML first
Missing connectors	Namespace not declared	Check xmlns declarations
Empty generated files	No flows in XML	Verify flow elements exist
DataWeave warnings	Complex patterns	Review and adjust manually
Duplicate flow warnings	Same name in files	Use unique flow names

16.3 LLM Validation Issues

Issue	Cause	Solution
Missing API key	No key provided	Enter key in AI Validation tab
Connection failed	Invalid key or network	Verify key, check internet
Takes too long	Large project/slow model	Use faster model (Groq, Gemini Flash)
Module not installed	LLM library missing	pip3 install anthropic openai

17. Glossary

Every technical term used in this documentation, explained in plain English:

API
A way for two programs to talk to each other over the internet.

API Key
A secret password that lets your application use a service (like an AI model).

Annotation
In Java, a marker starting with @ that adds behavior (e.g., @GetMapping).

Container
A portable package containing your application and everything it needs to run.

CORS
Cross-Origin Resource Sharing. Controls which websites can access your API.

DataWeave
MuleSoft's proprietary language for transforming data between formats.

Dependency
A library your project needs to work (listed in pom.xml for Java).

Docker
A platform for building and running containers (packaged applications).

Docker Compose
A tool for running multi-container applications with one command.

Endpoint
A specific URL path in an API (e.g., /api/customers).

Flask
A lightweight Python web framework used to build the migrator.

Flow
In MuleSoft, a sequence of steps triggered by an event.

Gunicorn
A production-grade Python web server for concurrent requests.

HTTP
The standard protocol for web communication.

JMS
Java Message Service. Standard for sending messages between apps.

JSON
A lightweight text format for data exchange.

Kafka
A distributed event streaming platform for high-throughput messaging.

LLM
Large Language Model. An AI system that understands and generates text/code.

Maven
A build tool for Java projects that manages dependencies.

Nginx
A high-performance web server used as a reverse proxy.

OAuth
An authentication standard for secure access delegation.

pom.xml
Maven build file listing all project dependencies.

REST
A standard architecture for building web APIs.

Spring Boot
Popular Java framework for production-ready web applications.

SSL/TLS
Security protocols that encrypt data in transit (HTTPS).

XML
Extensible Markup Language. Text format used by MuleSoft.

ZIP
Compressed file format that bundles multiple files together.

End of Documentation

MuleSoft to Spring Boot Migrator v1.0

March 2026